

Algorithms

Lecture #37

Syed Hamed Raza

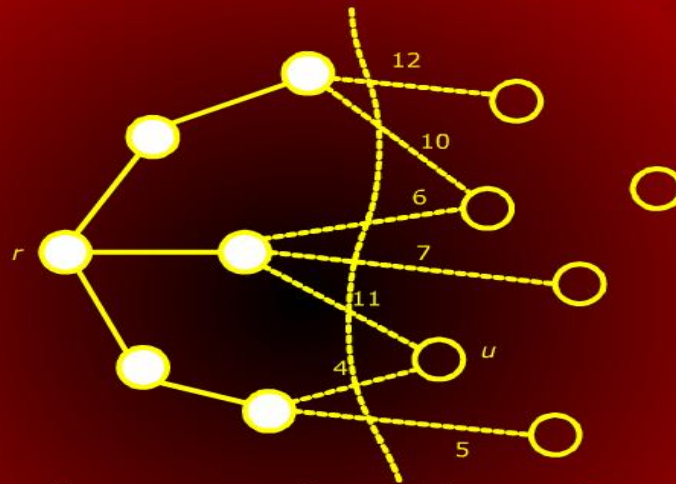
Prim's Algorithm

Prim's Algorithm

- Prim's algorithm builds the MST by adding leaves one at a time to the current tree.
- We start with a root vertex r ; it can be any vertex.
- At any time, the subset of edges A forms a single tree (in Kruskal's, it formed a forest).

Prime's Next Vertex to Add

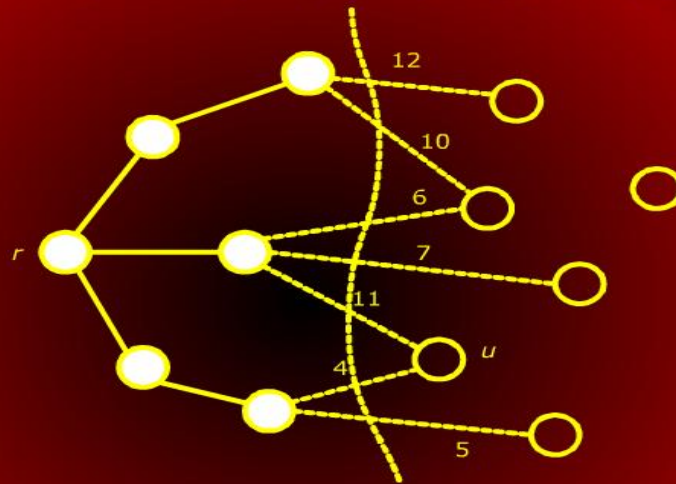
Prim's: Next Vertex to Add



Consider the set of vertices S currently part of the tree and its complement $(V - S)$.

Prime's Next Vertex to Add

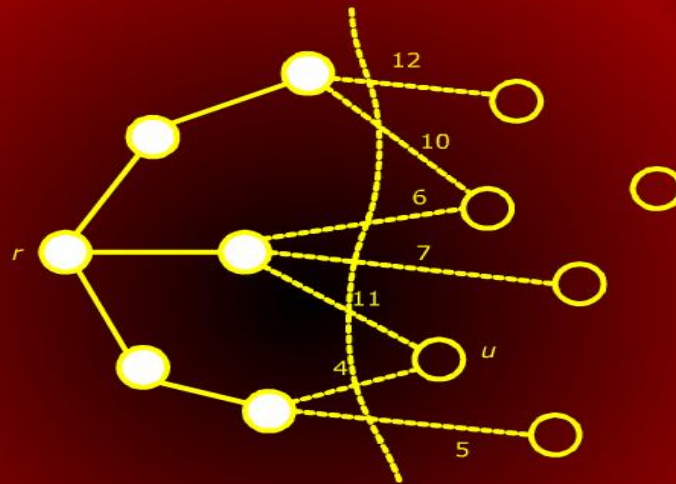
Prim's: Next Vertex to Add



We have cut of the graph.

Prime's Next Vertex to Add

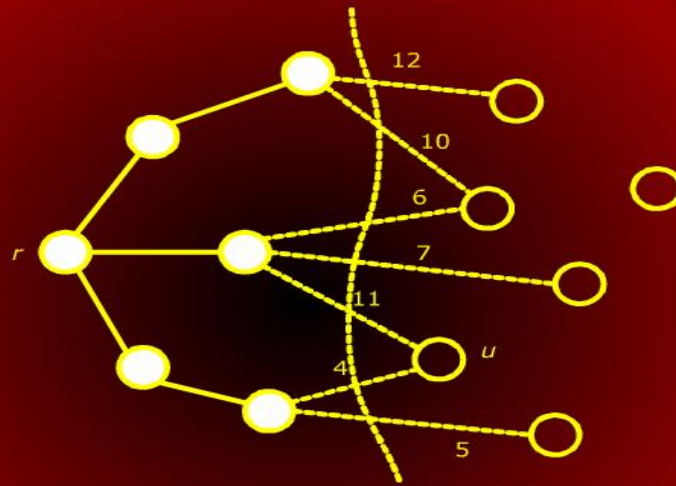
Prim's: Next Vertex to Add



Which edge should be added next?

Prime's Next Vertex to Add

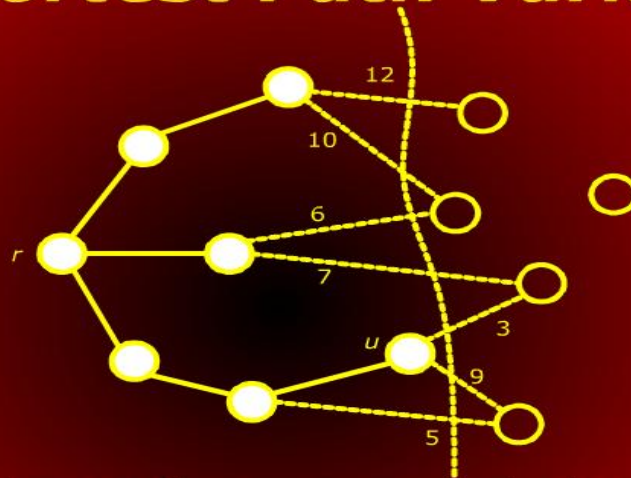
Prim's: Next Vertex to Add



The greedy strategy would be to add the lightest edge which in the figure is edge to 'u'.

Shortest Path Variants

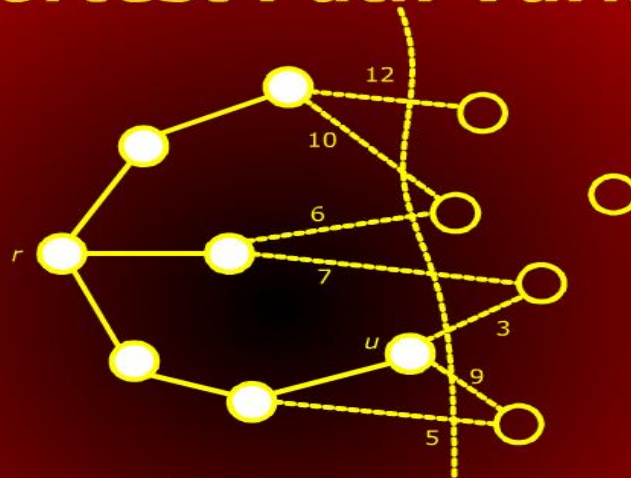
Shortest Path Variants



Some edges that crossed the cut are no longer crossing it and others that were not crossing the cut are.

Shortest Path Variants

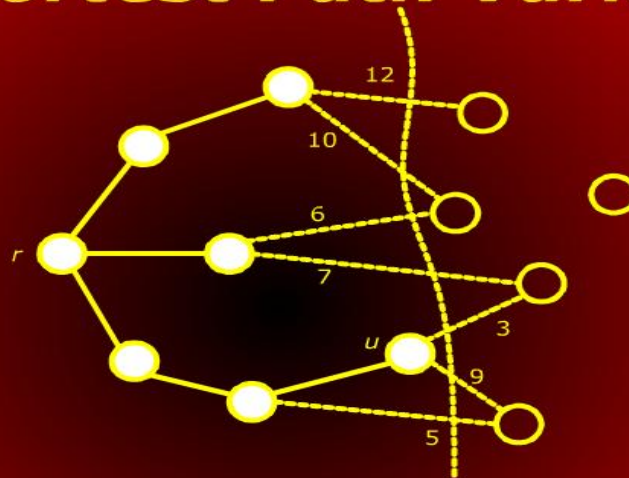
Shortest Path Variants



We need an efficient way to update the cut and determine the light edge quickly.

Shortest Path Variants

Shortest Path Variants



To do this, we will make use of a *priority queue*.

Prim's Algorithm: Priority Queue

Prim's Algorithm: Priority Queue

- What do we store in the priority queue?
- It may seem logical that edges that cross the cut should be stored since we choose light edges from these.
- Although possible, there is more elegant solution which leads to a simpler algorithm.

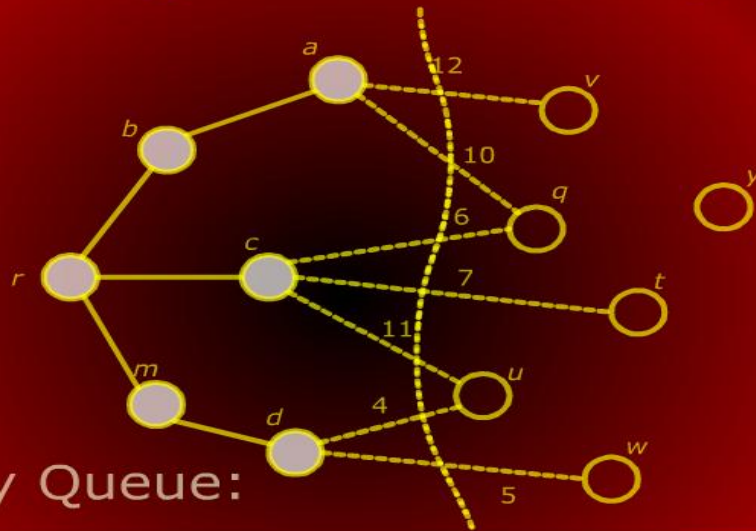
Prim's Algorithm: Priority Queue

Prim's Algorithm: Priority Queue

- For each vertex $u \in (V - S)$ (not part of the current spanning tree), we associate a key $\text{key}[u]$.
- The $\text{key}[u]$ is the weight of the lightest edge going from u to any vertex in S .
- If there is no edge from u to a vertex in S , we set the key value to ∞ .
- We also store in $\text{pred}[u]$ the end vertex of this edge in S .

Prim's Algorithm: Priority Queue

Prim's Algorithm: Priority Queue



Priority Queue:

$(u, 4) | (w, 5) | (t, 7) | (q, 6) | (v, 12) | (y, \infty)$

Prim's Algorithm

Prim's Algorithm

- We will also need to know which vertices are in S and which are not.
- To do this, we will assign a color to each vertex.
- If the color of a vertex is black then it is in S otherwise not.

Prim's Algorithm

Prim's Algorithm

PRIM((G,w, r))

```
1  for ( each  $u \in V$  )
2  do  $\text{key}[u] \leftarrow \infty$ ;  $\text{pq.insert}(u, \text{key}[u])$ 
3       $\text{color}[u] \leftarrow \text{white}$ 
4   $\text{key}[r] \leftarrow 0$ ;  $\text{pred}[r] \leftarrow \text{nil}$ ;
     $\text{pq.decrease\_key}(r, \text{key}[r])$ ;
5  while (  $\text{pq.not\_empty}()$  )
6  do  $u \leftarrow \text{pq.extract\_min}()$ 
7      for ( each  $u \in \text{adj}[u]$  )
```

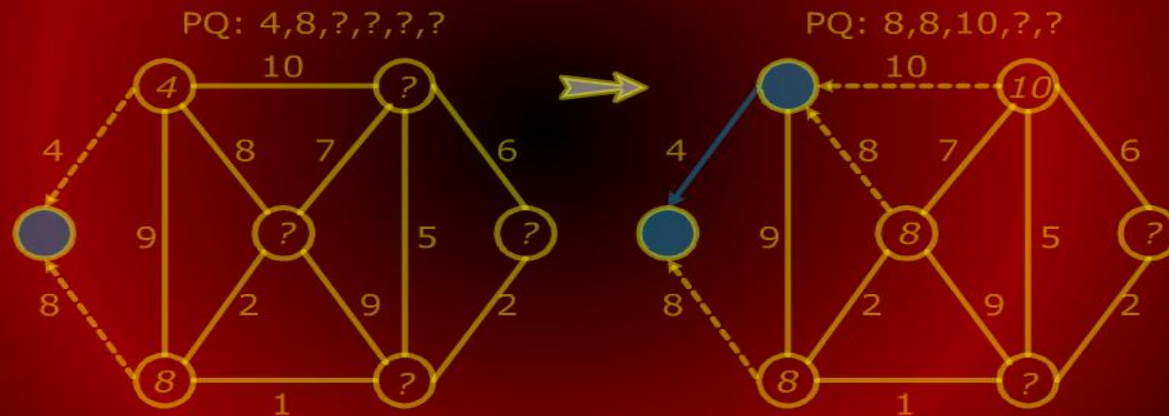

Prim's Algorithm

Prim's Algorithm

```
5  while ( pq.not_empty () )
6  do u ← pq.extract_min ()
7    for ( each u ∈ adj [u] )
8    do if ( color [v] == white ) and
        ( w (u, v) < key [v] )
9        then key [v] = w (u, v)
10       pq.decrease_key (v, key [v])
11       pred [v] = u
12    color [u] = black
```

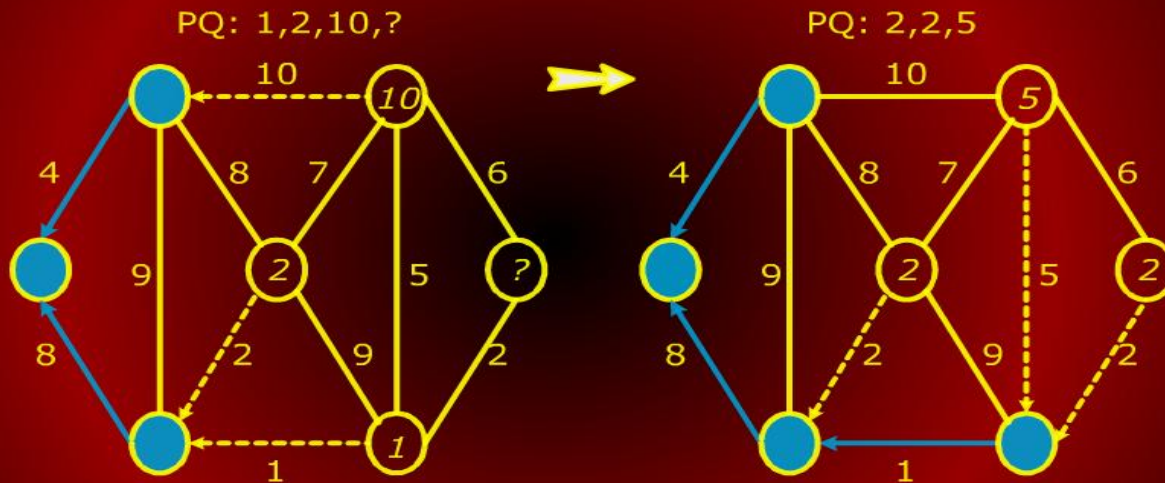
Prim's Algorithm Trace

Prim's Algorithm Trace



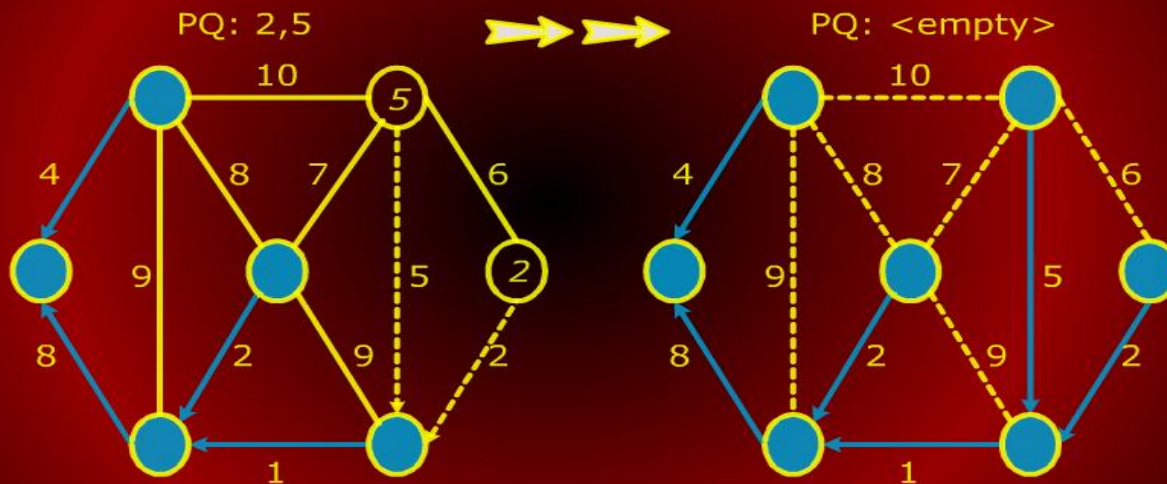
Prim's Algorithm Trace

Prim's Algorithm Trace



Prim's Algorithm Trace

Prim's Algorithm Trace



Prim's Algorithm Run Time Analysis

Prim's Algorithm Runtime Analysis

- It takes $O(\log V)$ to extract a vertex from the priority queue.
- For each incident edge, we spend potentially $O(\log V)$ time decreasing the key of the neighboring vertex.
- Thus the total time is $O(\log V + \deg(u) \log V)$.
- The other steps of update are constant time.

Prim's Algorithm Run Time Analysis

Prim's Algorithm Runtime Analysis

So the overall running time is

$$\begin{aligned} T(V, E) &= \sum_{u \in V} (\log V + \deg(u) \log V) \\ &= \log V \sum_{u \in V} (1 + \deg(u)) \\ &= (\log V)(V + 2E) \\ &= \Theta((V + E) \log V) \end{aligned}$$

Since G is connected, V is asymptotically

Prim's Algorithm Run Time Analysis

Prim's Algorithm Runtime Analysis

$$\begin{aligned} & \sum_{u \in V} \log V (1 + \deg(u)) \\ &= \log V \sum_{u \in V} (1 + \deg(u)) \\ &= (\log V)(V + 2E) \\ &= \Theta((V + E) \log V) \end{aligned}$$

Since G is connected, V is asymptotically no greater than E so this is $\Theta(E \log V)$, same as Kruskal's algorithm.

Shortest Path

Shortest Paths

Shortest Path

Shortest Path

- A motorist wishes to find the shortest possible route between Peshawar and Karachi.
- Given a road map of Pakistan on which the distance between each pair of adjacent cities is marked
- Can the motorist determine the shortest route?

Prim's Algorithm